

Topic 11.1

11.1: Linked Lists

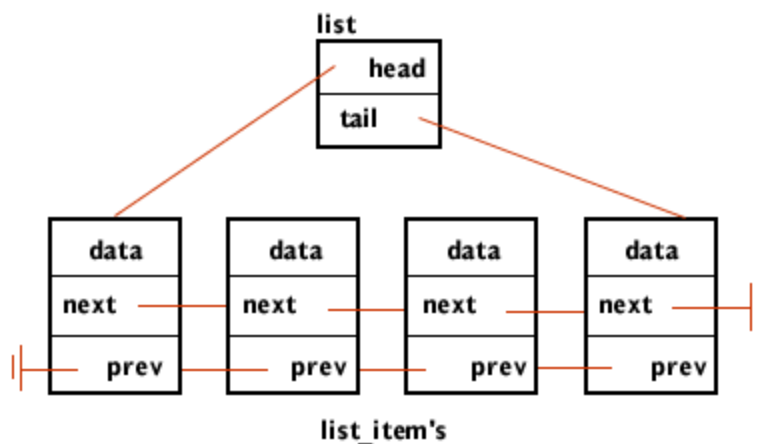
The following diagram illustrates the essential structure of a *linked list*.

We have a series of containers. Each container holds one piece of actual data, as well as pointers to the *next* and *previous* containers.

We also have an external structure known as the *list* that maintains pointers to the first and last containers in the list. Not shown (to preserve clarity) are pointers from each container to the list that contains that container.

A ListItem is an ADT that contains all the details of how containers are linked together. To create an actual list (see the main program), we derive a class from ListItem.

Lists can be ordered by one criteria. This criteria must be supplied by the client by overriding the pure virtual function [CompareByInsertKey](#). Lists can be searched on arbitrary criteria as provided by a comparison function.



```
// MODULE:      linklist.h
// PROGRAMMER:   Gary J. Blair
// LANGUAGE:     C++
// DATE:        97/08/04
//
// PURPOSE:
//   Provide a linked list capability.
//   Illustrate iterators.
//   Illustrate friend classes.
//   Illustrate ADTs.

#ifndef      linklist_h
#define      linklist_h

class List;           // Tracks the first and
                      // last items
                      // in a list.
class ListItem;       // Stores the actual
                      // items in the list.
class ListIterator;   // Used to loop over all
```

```

        // items in a list.
class IteratorException; // Thrown if iterator
                        // becomes out of date.

/* A COMPARATOR is a function that takes a
   pointer to an item in a list, and a pointer
   to a key we are searching for. It extracts
   the key of the item in the list and compares
   it against the search key, returning a value
   in the normal manner (e.g. strcmp). */

typedef int (*COMPARATOR)( ListItem* item_in_list,
                          void* search_key );

/* ListItem is an ADT that provides the
   capability to insert and remove items from a
   List. We use it by deriving a type from
   ListItem and defining a CompareByInsertKey
   member function. */

class ListItem // ADT
{
    friend class List;
    friend class ListIterator;

public:
    ListItem() {inserted=0; next=prev=NULL;
               plist=NULL;};

    /* CompareByInsertKey is invoked upon
       something derived from ListItem, which we
       are inserting into a List. It takes as an
       argument another ListItem, which is
       already in the list. It compares the keys
       of these two ListItems, returning a value
       in the usual manner for comparators
       (e.g. strcmp). */

    virtual int CompareByInsertKey
        ( ListItem* item_in_list ) = 0;

    // Clone the item.

    virtual ListItem* Clone() = 0;

    // Provides each ListItem the ability to
    // delete itself from the list which
    // contains it.

    void Delete();

private:
    ListItem* next;    // Ptr to next item.
    ListItem* prev;    // Ptr to previous item.
    int      inserted; // Item has been inserted
                  // in a list.
    List*    plist;    // Ptr to list
                  // containing this item.
};

/* List keeps track of the first and last items
   in a List. It also keeps a generation count,
   which is incremented each something is
   inserted or deleted. The purpose of this
   generation count is to enable iterators to
   detect when the list has changed while being
   iterated. */

```

```

class List
{
    friend class ListIterator;
    friend class ListItem;

public:
    List() {head=tail=NULL; generation=0;};

    // Insert new_item into the List.
    void Insert( ListItem* new_item );

    // Search the List for an item containing a
    // key equal to search_item_key, as
    // determined by c.
    ListItem* Find( void *search_item_key,
                    COMPARATOR c);

    // Delete deleted_item from the list.
    void Delete( ListItem* deleted_item );

    // Delete the item containing a key equal
    // to deleted_item_key, as determined by c.
    int Delete( void* deleted_item_key,
                COMPARATOR c );

    // Clone the list.
    virtual List* Clone();

private:
    ListItem* head; // First item in list.
    ListItem* tail; // Last item in list.
    unsigned long generation; // Generation number
                             // of the list.
                             // Provides validity
                             // check for
                             // iterators.

    unsigned long NextGeneration()
    {return ++generation;};
    unsigned long GetGeneration()
    {return generation;};
};

/* An iterator is an object that allows us
to iterate over a list by keeping track of
the next item in the list to process. We can
have many iterators for the same list active
simultaneously. */

class ListIterator
{
public:
    ListIterator( List& list_to_iterate );

    // Return a pointer to the next item
    // to be processed.

    ListItem* NextItemInList();

private:
    List* l; // List we are iterating over
    ListItem* next_item; // Pointer to next item

```

```

        // to be processed
    unsigned long generation; // Records
                               // l->generation at
                               // time when iterator
                               // is created.
};

/* This exception is thrown when we attempt to
   use an old iterator. An iterator is old if
   the list it iterates has been modified (an
   insert or delete has occurred) since the
   iterator was created. This will be detected
   the iterator's generation, recorded when the
   iterator is constructed, will not match the
   current generation of the list. */

class IteratorException
{
public:
    IteratorException( List& l )
    { iterated_list = &l; };

    List* iterated_list; // List exception
                        // occurred on.
};

#endif

// MODULE:      linklist.cpp
// PROGRAMMER:   Gary J. Blair
// LANGUAGE:     C++
// DATE:        97/08/04
//
// PURPOSE:
//   Provide a linked list capability.
//   Illustrate iterators.
//   Illustrate friend classes.
//   Illustrate ADTs.

#include <iostream.h>
#include <string.h>
#include "linklist.h"

void ListItem::Delete()
{
    plist->Delete(this);
}

void List::Insert( ListItem *new_item )
{
    ListItem *search; // Ptr to search items
                     // in list.
                     // Want to find first item
                     // in list with a
                     // key >= new_item's key.

    if (! head) // List is currently empty.
    {
        head = tail = new_item;
        new_item->next = new_item->prev = NULL;
    }
    else // Find first item larger than
        // inserted item
    {
        for (search=head; search;
            search=search->next)

```

```

        if (new_item->CompareByInsertKey(search)
            < 0)
            break;

        if (search==head) // Inserting new
                           // first item.
        {
            head = new_item;
            new_item->next = search;
            search->prev = head;
            new_item->prev = NULL;
        }
        else if (! search) // Means insert as
                           // new last item.
        {
            tail->next = new_item;
            new_item->prev = tail;
            new_item->next = NULL;
            tail = new_item;
        }
        else
        {
            new_item->prev = search->prev;
            search->prev->next = new_item;
            new_item->next = search;
            search->prev = new_item;
        }
    }
    new_item->inserted = 1;
    new_item->plist = this;
    NextGeneration();
}

ListItem* List::Find( void *search_item_key,
                     COMPARATOR c)
{
    ListItem *search; // Ptr to traverse
                      // each item in list.

    for (search=head; search; search=search->next)
        if (! c(search,search_item_key))
            return search;
    return NULL;
}

void List::Delete( ListItem *deleted_item )
{
    if (head == deleted_item)
        head = deleted_item->next;
    else
        deleted_item->prev->next
            = deleted_item->next;

    if (tail == deleted_item)
        tail = deleted_item->prev;
    else
        deleted_item->next->prev
            = deleted_item->prev;

    deleted_item->next = NULL;
    deleted_item->prev = NULL;
    deleted_item->inserted = 0;
    deleted_item->plist = NULL;
    NextGeneration();
}

int List::Delete( void *deleted_item_key, COMPARATOR c )

```

```

{
    ListItem *search; // Ptr to traverse
                        // each item in list.

    if (search=Find(deleted_item_key,c))
    {
        Delete(search);
        return 1;
    }
    else
        return 0;
}

List* List::Clone()
{
    List      *newList; // Ptr to duplicate list.
    ListItem  *p;        // Ptr to traverse
                        // original list.
    ListItem  *pdup;     // Ptr to duplicate of p.
    ListIterator *iter;  // Iterator over original list.

    newList = new List;
    if (! newList)
        return NULL;

    iter = new ListIterator(*this);
    while (p=(iter->NextItemInList()))
    {
        pdup = p->Clone();
        if (! pdup)
            return newList;
        newList->Insert(pdup);
    }
    delete iter;
    return newList;
}

ListIterator::ListIterator( List& list_to_iterate )
{
    l = &list_to_iterate;
    next_item = l->head;
    generation = l->GetGeneration();
}

ListItem* ListIterator::NextItemInList()
{
    ListItem *rv; // Return value - ptr to
                  // next item.

    // if (generation != l->GetGeneration())
    // The list changed!
    // throw IteratorException(*l);

    rv = next_item;
    if (next_item)
        next_item = next_item->next;
    return rv;
}

// MODULE:      person.h
// PROGRAMMER:   Gary J. Blair
// LANGUAGE:     C++
// DATE:        97/08/04
//
// PURPOSE:
//   Store information about a person.

```

```

// Derive a concrete type from the ADT ListItem.

#ifndef person_h
#define person_h

#include <iostream.h>
#include <string.h>
#include "linklist.h"

class Person : public ListItem
{
public:
    Person( char *s, unsigned long id ) {strcpy(name,s); ssn=id;};
    virtual int CompareByInsertKey
        ( ListItem* item_in_list );
    virtual void print();
    virtual ListItem* Clone();

    char name[50]; // Person's name.

private:
    unsigned long ssn; // Social security number.

    friend int CompareSSN
        (ListItem* item_in_list, void* search_key);
};

#endif

// MODULE:      person.cpp
// PROGRAMMER:   Gary J. Blair
// LANGUAGE:     C++
// DATE:        97/08/04
//
// PURPOSE:
//   Store information about a person.
//   Derive a concrete type from the ADT ListItem.

#include "person.h"

// Clone the item.

ListItem* Person::Clone()
{
    Person *p;

    p = new Person(name,ssn);
    return (ListItem *) p;
}

int Person::CompareByInsertKey
    ( ListItem* item_in_list )
{
    return strcmp(this->name,
        ((Person *)item_in_list)->name);
}

void Person::print()
{
    cout << name << ":" << ssn << endl;
}

int CompareSSN( ListItem* item_in_list, void* search_key )
{
    unsigned long left
        = ((Person *)item_in_list)->ssn;
    unsigned long right

```

```

        = *((unsigned long *) search_key);
    return (left < right) ? -1 :
           (left > right) ? 1 :
           0 ;
}

// MODULE:      llmain.cpp
// PROGRAMMER:   Gary J. Blair
// LANGUAGE:     C++
// DATE:        97/08/04
//
// PURPOSE:
//   Demonstrate Lists.

#include <iostream.h>
#include <string.h>
#include "linklist.h"
#include "person.h"

void main(void)
{
    Person      *p, *p1,*p2,*p3,*p4,*p5,*p6,*p7;
    List        l;
    ListIterator *iter;
    unsigned long ssn;
    List        *cloned_list;

    p1 = new Person("Kmetz, Rhonda",889922336);
    l.Insert(p1);
    p2 = new Person("Jones, Tom", 122334455);
    l.Insert(p2);
    p3 = new Person("Phillips, Bob",545667887);
    l.Insert(p3);
    p4 = new Person("Adams, Don", 999997777);
    l.Insert(p4);
    p5 = new Person("Zwakenberg, Rich", 888777666);
    l.Insert(p5);
    p6 = new Person("Blair, Gary", 777666321);
    l.Insert(p6);
    p7 = new Person("Johnson, Bill", 221835412);
    l.Insert(p7);

    iter = new ListIterator(l);
    while (p=(Person *)(iter->NextItemInList()))
        p->print();
    delete iter;

    cloned_list = l.Clone();
    cout << "The cloned list:\n";
    iter = new ListIterator(*cloned_list);
    while (p=(Person *)(iter->NextItemInList()))
        p->print();
    delete iter;

    l.Delete(p2);
    p4->Delete();
    ssn = 888777666;
    l.Delete(&ssn,CompareSSN);

    cout
    << "\nAfter deleting Jones, Adams, and Zwakenberg
        the list is\n\n";
    iter = new ListIterator(l);
    while (p=(Person *)(iter->NextItemInList()))
        p->print();
    delete iter;

    ssn = 889922336;

```

```
p = (Person *) (l.Find(&ssn, CompareSSN));  
cout  
<< "\nResult of searching for SSN 889922336 is"  
<< endl;  
p->print();  
}
```

Program output: llmain.cpp

```
Adams, Don:999997777  
Blair, Gary:777666321  
Johnson, Bill:221835412  
Jones, Tom:122334455  
Kmetz, Rhonda:889922336  
Phillips, Bob:545667887  
Zwakenberg, Rich:888777666  
The cloned list:  
Adams, Don:999997777  
Blair, Gary:777666321  
Johnson, Bill:221835412  
Jones, Tom:122334455  
Kmetz, Rhonda:889922336  
Phillips, Bob:545667887  
Zwakenberg, Rich:888777666  
After deleting Jones, Adams, and Zwakenberg the list is:  
Blair, Gary:777666321  
Johnson, Bill:221835412  
Kmetz, Rhonda:889922336  
Phillips, Bob:545667887  
Result of searching for SSN 889922336 is:  
Kmetz, Rhonda:889922336
```

[Back to Module 11 \(https://onlinelearning.berkeley.edu/courses/2291660/pages/module-11\)](https://onlinelearning.berkeley.edu/courses/2291660/pages/module-11)